Multi-Platform Container Rebuild Report

Multi-platform container rebuild report

Project: ExSOIL NSF Prototype **Period:** May-June 2026 (updated June 10) **Prepared by:** Steven Wangen, UW-Madison Data Science Institute

---

Section 1: Overview

What was rebuilt and why

This project runs the NCAR CTSM (Community Terrestrial Systems Model) land modeling framework inside a Docker container. Researchers pull the container image, launch it, and open JupyterLab in their browser to run NEON tower site simulations and analysis notebooks. The container packages everything needed: an operating system, scientific Fortran/C libraries, the CTSM source code and build system, a full Python scientific stack (NumPy, xarray, cartopy, etc.), and JupyterLab itself.

The original container image (`escomp/cesm-lab-neon`) was built in October 2022 on CentOS 8 (which lost security support in December 2021) and only supported Intel/AMD processors. It shipped Python 3.7 (end-of-life since June 2023) and hundreds of outdated packages. Most critically, it could not run natively on Apple Silicon Macs (M1/M2/M3/M4), which use a different processor architecture called ARM64. Team members on these machines had to run the container through a translation layer that was 2-3x slower and crashed frequently during computationally intensive operations like cartopy map rendering and model compilation.

Additionally, the NEON tower site workflow was non-functional. Investigation revealed that NEON support was never part of any CESM 2.x release; the original container used an undocumented custom build that grafted NEON support from a development branch. This graft broke as the components diverged.

The container was rebuilt from scratch on Ubuntu 24.04 with standalone CTSM 5.4.043, native support for both processor architectures, modern packages, and a reproducible build system.

What multi-platform support means

A Docker image is compiled code, and compiled code is specific to a processor architecture. An image built for Intel/AMD (called "amd64" or "x86_64") contains machine instructions that ARM processors cannot execute directly, and vice versa. When you run an amd64 image on an ARM Mac, Docker uses a technology called QEMU emulation to translate instructions on the fly. This works, but it is slow, uses more memory, and can trigger subtle bugs in complex software.

Multi-platform support means publishing a single image tag (like `latest`) that actually contains two architecture-specific images behind the scenes, organized in what Docker calls a "manifest list." When someone runs `docker pull`, their Docker client automatically selects the correct image for their machine. An engineer on an Intel Linux server gets the amd64 image. An engineer on an M3 MacBook gets the arm64 image. Neither needs to know or care about the other architecture.

The approach

Rather than cross-compiling the old image, the team rebuilt the entire stack on a new foundation. The original image compiled five scientific libraries (MPICH, HDF5, NetCDF-C, NetCDF-Fortran, PNetCDF) from Fortran and C source code inside the container, a process that took 30-45 minutes and was architecture-specific. The rebuild replaced all of this with pre-built binary packages from conda-forge, a community package repository that publishes builds for both amd64 and arm64. This reduced build time to about five minutes and eliminated the most complex source of architecture-dependent code.

The project also migrated from the full CESM 2.2.2 (which lacked native NEON support) to standalone CTSM 5.4.043 (which includes the NEON workflow natively), trimmed the Python environment from approximately 400 packages to 35 direct dependencies, introduced reproducible lockfiles for both architectures, and created a three-tier automated test suite that validates the container on each platform.

What changed for downstream consumers

Nothing, operationally. The image is published to the same GitHub Container Registry address. `docker pull` and `docker run` commands are unchanged. On Apple Silicon Macs, the container now runs natively instead of through emulation, so it is faster, more stable, and does not crash during heavy computation. The container also now supports end-to-end NEON simulations (create case, build model, run, archive, analyze) which was not possible with the previous image.

Three products are in play, each nesting inside the next. **CESM** (Community Earth System Model) is the full coupled Earth system: atmosphere (CAM), ocean (POP/MOM6), sea ice (CICE), ice sheets (CISM), waves (WW3), and land (CLM), all exchanging energy and moisture through a coupler. It is designed for global climate projections. **CTSM** (Community Terrestrial Systems Model) is a self-contained release of just the land portion. It includes CLM (the land simulation code), CIME (the build and case management system), DATM (a data atmosphere that reads weather from files instead of simulating it), a coupler (CMEPS), and the NEON tower workflow (48 pre-configured site setups). CIME and CLM share common ancestry with CESM (they are developed in the same repositories), but CTSM is packaged and released independently by NCAR as its own product. It is what NCAR recommends for single-site land experiments.

The original **ExSOIL** container (`escomp/cesm-lab-neon`, October 2022) was not built on CTSM. It was built on the full CESM 2.2 framework and carried the complete source code for the atmosphere, ocean, sea ice, ice sheet, and wave models (~3-4 GB) even though none of them were used. The NEON tower workflow did not exist in any CESM 2.x release, so it was grafted in from a development branch of CTSM that predated any official release. That graft broke as the components diverged. The container ran on CentOS 8 (end-of-life), Python 3.7 (end-of-life), and only supported Intel/AMD processors. This report covers the rebuild, which replaces the CESM base

with standalone CTSM 5.4.

Section 2: Technical detail

Base image selection

Decision: `ubuntu:24.04` (untagged digest, pulled at build time).

Rationale (documented in ADR-0001): The upstream image used CentOS 8 (EOL). Six alternatives were evaluated:

Option	Multi-arch	Compressed size	EOL	Community fit
Ubuntu 24.04	amd64, arm64 + 4 more	~29 MB	Apr 2029 (free), 2036 (ESM)	Pangeo, Jupyter Stacks
AlmaLinux 9	amd64, arm64	~30 MB	May 2032	CERN, Fermilab
Rocky Linux 9	amd64, arm64	~44 MB	May 2032	CIQ HPC
Debian 12	amd64, arm64	~47 MB	Jun 2028 (LTS)	Less common for HPC
jupyter/scipy-notebook	amd64, arm64	~1.5 GB	Follows Ubuntu	Jupyter community
condaforge/miniforge3	amd64, arm64	~148 MB	Follows Ubuntu	Pangeo pattern

Ubuntu was selected because both Pangeo and Jupyter Docker Stacks have standardized on it, it has the longest free support window, and its `apt` ecosystem has the best-documented path for Fortran/C scientific library compilation.

Dockerfile structure

The Dockerfile uses a three-stage build, all stages deriving from the first:

```
Stage 1 ("base")   FROM ubuntu:24.04    Conda environment + scientific stack
Stage 2 ("ctsm")   FROM base            CTSM source + machine config
Stage 3 ("app")    FROM ctsm            Project notebooks + analysis modules
```

This is a linear chain, not a parallel multi-stage build. There is no separate builder stage; compilation tools (gfortran, gcc, cmake) remain in the final image because CTSM's `case.build` workflow requires them at runtime.

Stage 1: base

1. `apt-get` installs system packages: compilers (`build-essential`, `gfortran`), build tools (`cmake`, `m4`, `make`), version control (`git`, `subversion`), Perl XML processing (`perl`, `libxml-libxml-perl`), and utilities.
2. Miniforge installer detects architecture automatically:

```
RUN wget -q0 /tmp/miniforge.sh \
    "https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Linux-$(
```

3. Conda environment installed from explicit lockfiles with architecture selection at build time:

```
RUN ARCH=$(uname -m) \
    && if [ "$ARCH" = "aarch64" ]; then LOCKFILE=/tmp/conda-linux-aarch64.lock; \
       else LOCKFILE=/tmp/conda-linux-64.lock; fi \
    && mamba install --name base --file "$LOCKFILE" --yes --quiet
```

4. Optional Dask layer controlled by build arg (`INSTALL_DASK_DISTRIBUTED=false`).

Stage 2: ctsm

1. Clones CTSM 5.4.043 and runs `git-fleximod update` to pull component repositories (CLM, CIME, CDEPS, CMEPS, MOSART, PIO).
2. Overlays container machine configs with conda-forge library paths (`$CONDA_PREFIX` instead of `/usr/local`).
3. Patches the NEON usermods to remove `MPILIB=mpi-serial` (conflicts with conda-forge MPICH at runtime).
4. `chown -R user:ctsm` transfers ownership of the CTSM tree to the runtime user (required because CIME creates build artifacts in-tree).

Stage 3: app Copies project-specific code: analytics modules, run_neon_v2 wrapper, notebooks. Sets `PYTHONPATH` to include CTSM Python module directories. Configures git user for CIME (which commits during case.build).

Cross-compilation strategy

There is no cross-compilation. Both architectures build natively (amd64 on native hardware, arm64 under QEMU emulation on the CI runner). This is viable because:

- **No source compilation of Fortran/C libraries in the image build.** All scientific libraries come from conda-forge pre-built binaries.
- **The Miniforge installer and conda-forge lockfiles are architecture-specific.** Each architecture gets its own resolved dependency graph.
- **CTSM Fortran compilation happens at runtime (`case.build`), not during the Docker build.** The image ships source code and a toolchain; the user compiles when they create a case. Tested and confirmed working on native arm64 in ~100 seconds.

Dependency management and reproducibility

The project uses a three-file system documented in ADR-0003:

1. `environment.yml` (human-maintained): ~35 direct dependencies with loose version constraints. Python >=3.11 (no upper bound; CTSM 5.4's CIME 6.1 supports Python 3.12+). cmake with no upper bound (CTSM's PIO 2.6 is compatible with cmake 4.x).
2. `conda-lock.yml` (generated): Unified lockfile produced by `conda-lock lock` for both platforms. Contains exact versions and hashes.
3. `conda-linux-{64,aarch64}.lock` (generated): Explicit lockfiles rendered from `conda-lock.yml` via `conda-lock render`.

Calibration and validation

CLM ships with default parameter values (soil thermal conductivity, hydraulic conductivity, root depth distribution, stomatal conductance, and hundreds more) that are tuned to global averages. The soil at Konza Prairie is not the same as the soil at a NEON site in Alaska or Florida. The core research question is whether those defaults are adequate for a given site, and if not, which parameters need adjustment.

This is where the Kalman filter calibration comes in. It takes the mismatch between CLM's predictions and the NEON terrestrial observations and works backward: if the model consistently predicts soil temperature 2 degrees too warm, which parameter adjustments (within physically plausible bounds) would bring the prediction closer to what the sensors measured? The output is a set of site-specific parameter values that make CLM better match that particular location. The scientific value is twofold: better predictions at that site, and insight into what CLM gets systematically wrong (if thermal conductivity always needs adjustment at grassland sites, that points to a structural issue in how CLM represents grassland soils).

An open question is what the actual workflow target is. There are two distinct goals, and they require different levels of validation:

- **Diagnostic comparison:** Run CLM with default parameters, compare output against NEON observations, and assess where the model is right and where it falls short. This is useful on its own for understanding model behavior, but it treats CLM as a fixed object being evaluated.
- **Kalman filter calibration:** Use the model-observation mismatch to iteratively adjust CLM's parameters, producing a calibrated model tuned to a specific site. This is the more ambitious goal and the one the `analytics_modules` were built for. If this is the primary use case, the diagnostic comparison is a means to an end (quantifying the misfit that drives calibration), not an end in itself.

We need to confirm with Jingyi which of these is the target before investing in notebook validation. If the goal is calibration, the diagnostic visualizations are intermediate outputs in that pipeline rather than standalone deliverables.

Model-observation comparison

Regardless of the end goal, the comparison between CLM output and NEON observations is the foundation. "Evaluating" means comparing CLM's predictions (soil temperature, soil moisture, carbon fluxes, heat fluxes) against the actual measurements recorded by NEON sensors at the same site, over the same time period. The model says soil temperature at 10 cm was 12.3 degrees

on January 15; the NEON sensor at 10 cm recorded 11.8 degrees. That comparison, repeated across variables, depths, and time, quantifies where the model is accurate and where it diverges.

The comparison uses standard diagnostic visualizations (time series, seasonal cycles, depth profiles, scatter plots, Taylor diagrams) that each answer a different question about model performance.

Diagnostic visualizations (details)

Diagnostic	What it shows	What failures look like
Time series	CLM's predicted value plotted alongside the NEON sensor measurement over the same period. The most intuitive view.	Model diverges during specific seasons or events (e.g., always too warm in July, misses freeze events).
Diurnal cycle	Half-hourly data averaged by time of day. Does CLM capture the daily heating/cooling pattern?	Amplitude wrong (too hot at midday, too cold at night) or timing shifted (peak an hour late).
Seasonal cycle	Monthly means over the simulation period. Captures freeze/thaw timing, growing season onset, summer drought stress.	Spring thaw too early, growing season too long, winter temperatures biased.
Depth profiles	Soil temperature or moisture as a function of depth, model layers vs sensor depths.	Surface close but deeper layers diverge. Common when soil thermal properties are miscalibrated.
Scatter plots	Model vs observation with a 1:1 line. Gives correlation (R2), bias, and RMSE in a single view.	Systematic offset from the 1:1 line (consistent bias). Wide scatter (poor correlation).
Taylor diagrams	Polar plot showing correlation, standard deviation ratio, and centered RMS difference for multiple variables simultaneously.	Dots far from the reference point. Lets you compare performance across variables at a glance.
Residual analysis	Model minus observed over time. Reveals systematic patterns vs random error.	"Always 2 degrees too warm in summer" is a systematic bias. Random scatter around zero means unbiased.

When the model disagrees with observations, the natural question is whether the disagreement is meaningful or within the range of uncertainty. Both sides of the comparison carry uncertainty: the observations have sensor precision and representativeness limitations, and the model has parameters that could plausibly take a range of values. Understanding the uncertainty envelope around both

the predictions and the measurements is what separates "the model is wrong" from "the model is within the range of what we'd expect given what we don't know." This matters for the calibration step: you only want to adjust model parameters to fix real biases, not to overfit to noise in the observations.

Uncertainty quantification (details)

Observation uncertainty. NEON sensor measurements have their own uncertainty. Temperature sensors have instrument precision (~0.1 degrees C), but the bigger issue is representativeness: a sensor at one spot in a prairie may not represent the grid cell CLM is simulating. NEON publishes quality flags and uncertainty estimates with their data products, particularly for the eddy covariance flux measurements (which have well-documented random and systematic error budgets).

Parameter uncertainty. CLM has hundreds of parameters (soil thermal conductivity, root distribution, stomatal conductance coefficients). Each has a physically plausible range. The Kalman filter calibration in `analytics_modules` is designed to explore this: it adjusts parameters within their physical bounds and quantifies how much the prediction changes. The posterior parameter distribution gives a measure of parametric uncertainty.

Ensemble perturbation. Run CLM multiple times with perturbed parameters or perturbed forcing data. The spread of the ensemble is the uncertainty band. This is what the Design_Hub_v2 notebook's perturbation experiments are for: vary inputs systematically and compare the resulting output distributions.

Structural uncertainty. CLM makes simplifying assumptions (number of soil layers, how roots take up water, biogeochemistry parameterizations). Quantifying this requires comparing CLM against a different land model entirely, which is outside the scope of ExSOIL.

Current validation status

Capability	Tool	Status
Run CLM, produce output	CTSM + NEON workflow	Working
Read output with Python	xarray	Working
Fetch NEON observations for comparison	`download_eval_files`	Not yet validated
Time series / scatter / profile diagnostics	Modeling_Hub notebook	Needs validation
Kalman filter calibration	analytics_modules	Needs validation
Perturbation experiments	Design_Hub_v2 notebook	Needs validation

The infrastructure to run simulations and read output is in place. The next milestone is connecting the observation data and validating the diagnostic notebooks against real output from this container.

The migration from CESM 2.2.2 to standalone CTSM 5.4.043 resolved most of the original build

compatibility issues. A handful of fixes remain in the container machine config for arm64 compilation and conda-forge library compatibility.

Compatibility fixes (details)

Issue	Root cause	Fix	Location
`rdtsc` asm error in GPTL	x86-only instruction (`HAVE_NANOTIME`)	Removed `-DHAVE_NANOTIME -DBIT64` from GPTL CPPDEFS	`container.cmake`
MPI type mismatch	GCC 10+ strict type checking	`-fallow-argument-mismatch`	`container.cmake`
BOZ hex literal error	GCC 10+ strict BOZ checking	`-fallow-invalid-boz`	`container.cmake`
`_FillValue` macro renamed	NetCDF-C renamed to `NC_FillValue`	`-D_FillValue=NC_FillValue`	`container.cmake`
ESMFMKFILE not set	NUOPC coupling requires ESMF	Set in `config_machines.xml` environment	`config_machines.xml`
MPILIB=mpi-serial conflict	Conda-forge MPICH on library path conflicts with CIME serial stubs	Patched NEON usermods to remove override	`Dockerfile`

Issues resolved by the CTSM migration (no longer apply): PIO2 source patches, Python 3.12 pin, cmake <4 pin, bundled `six.py` conflicts, `import imp` deprecation.

End-to-end simulation validation

Beyond the automated test suite, a full NEON simulation was run inside the container:

1. Pre-download global input data (~6 GB, 31 files) via the `pre-download-inputdata.sh` script with retries and server fallback (GDEX, SVN, FTP)
2. Create a KONZ transient case via `run_tower --neon-sites KONZ --run-type transient`
3. Build the Fortran model (~100 seconds, produces `cesm.exe`)
4. Download NEON tower forcing data (84 monthly files from Google Cloud Storage)
5. Run via `case.submit --no-batch` (CIME manages MPI execution)
6. Archive output (history files, restart files, logs)
7. Read output with xarray: 31 CLM variables, 48 half-hourly time steps

Output includes soil temperature (TSOI), soil moisture (H2OSOI), and sensible heat flux (FSH), which are the variables needed for model-observation comparison.

Known issues and concerns

Data hosting reliability NCAR's new GDEX data server (`osdf-data.gdex.ucar.edu`) uses a 3-hop redirect chain that intermittently returns empty responses or times out. Our pre-download

script works around this with 5 retries per file and fallback to SVN and FTP servers. This achieves 100% success across 31 files, but initial data setup still takes 10-15 minutes and any single download can stall for up to 5 minutes before a retry kicks in. The underlying reliability problem is on NCAR's infrastructure; we have no control over it.

Mitigation options: Cache the ~6 GB of global input data on university S3 (eliminates NCAR dependency entirely), or bundle it in the container image (increases image size from ~7 GB to ~13 GB but removes the download step).

Development tag, not an official release We are running CTSM 5.4.043, which is a **development tag** on CTSM's main branch, not an official release. The most recent official release (ctsm5.4.002, December 2025) shipped before NCAR updated its data server configuration files, so it points at old servers where the data no longer exists. The 5.4.043 tag was the earliest tag that uses the new GDEX server correctly.

This means we are tracking a moving target. If NCAR publishes a breaking change on main, our tag will not be affected (tags are immutable), but any future upgrade will need careful testing. When NCAR publishes a 5.4.x release with working data server config, we should pin to that instead.

NEON tower data temporal coverage NEON tower forcing data is available through approximately September 2024 for most sites (84 monthly files from January 2018). The last three months of 2024 (October, November, December) are not yet available on NEON's data portal. The NEON usermods default to `DATM_YR_END=2022` for transient runs, so this is not a blocker for most simulations, but researchers who want to run through 2024 will see missing-file warnings for those months.

MPI workaround, not an upstream fix The NEON workflow defaults to `MPILIB=mpi-serial` (CIME's built-in serial MPI stub). In a conda-forge environment, the real MPICH shared libraries are always on the linker path, and the two conflict at runtime. Our fix patches the NEON usermods in the Dockerfile to remove the `mpi-serial` override so cases use real MPICH. This is a container-level workaround, not a fix in the CTSM source. Any CTSM upgrade will need the same patch reapplied.

amd64 not tested end-to-end All local testing (90-test suite, case.build, simulation run) was on native arm64 (Apple Silicon). The amd64 path builds successfully in CI and the lockfiles exist, but the full test suite and simulation have not been exercised on amd64 hardware.

Science notebooks not validated with simulation output The Modeling_Hub, Design_Hub_v2, and Data_Hub notebooks are present in the container but have not been tested against real CLM output from this container. They were developed against output from the previous container (CESM 2.x, different variable naming conventions). The Getting_Started notebook has been validated.

No upstream issue filed We have not yet reported the data server mismatch or the `mpi-serial`/conda-forge conflict to ESCOMP/CTSM on GitHub. A draft issue exists at `docs/ctsm-issue-draft.md` but has not been submitted. Understanding whether these are known issues or novel findings would help determine whether our workarounds are temporary or long-term.

Open questions

For Jingyi (research use cases)

- Which NEON sites are needed? We support all 48, but validation effort should focus on priority sites.
- What simulation periods? The 1-day KONZ run works; are months, years, or specific date ranges needed?
- Which output variables matter most? We produce 31 CLM variables; the analysis may only need a subset.
- What does the current model-observation comparison workflow look like? Does it match the diagnostic patterns we described (time series, depth profiles, Taylor diagrams), or are there different approaches?

For the team (infrastructure decisions)

- **Delivery model:** Is a distributable Docker container sufficient (researchers pull and run locally), or do we want to host a shared instance (e.g., JupyterHub on campus infrastructure or cloud)? A hosted option eliminates the ~6 GB data download and local Docker requirement, but adds hosting cost and maintenance. A container-only approach is simpler but puts setup burden on each user.
- Do we cache global input data on campus S3, or bundle it in the container image (~7 GB to ~13 GB)?
- Do we file an upstream issue with ESCOMP/CTSM about the data server mismatch and mpi-serial conflict, or wait until we better understand whether these are known issues?
- When do we open the PR to merge to main and publish the image?
- Do we need amd64 validation before merging, or is arm64-only sufficient for now?

For NCAR / upstream (unresolved externalities)

- When will a stable CTSM 5.4.x release ship with working GDEX server config so we can move off the dev tag?
- Is the GDEX CDN reliability being addressed, or is this the new normal?
- Are the last 3 months of 2024 NEON tower data (Oct-Dec) going to be published?

Next steps

1. **Review use cases with Jingyi.** Confirm which NEON sites, simulation periods, output variables, and analysis patterns are needed so subsequent work is guided by actual research requirements.
2. **Multi-day and multi-site validation.** Extend the 1-day KONZ run to longer periods and additional NEON sites (prioritize based on Jingyi's input).
3. **Science notebook validation.** Connect Modeling_Hub to simulation output for model-data comparison. Integrate NEON terrestrial observations via `download_eval_files`. Validate the Kalman filter calibration and perturbation experiment workflows.
4. **Data caching.** Cache global input data on university S3 or bundle it in the container to eliminate the download bottleneck.

5. **CI/CD test integration.** Wire the 90-test suite into GitHub Actions for both architectures.
6. **Pin to a stable CTSM release.** When NCAR publishes a 5.4.x release with working GDEX config, upgrade from the dev tag.
7. **File upstream issue.** Report the data server and mpi-serial findings to ESCOMP/CTSM.
8. **PR and merge.** Open pull request from the feature branch and publish the updated image.

Testing strategy

A three-tier pytest framework runs inside the container via `tests/run_container_tests.sh`:

Tier	Tests	Runtime	What it validates
0	63	~4s	Python imports, compiler presence, Fortran+MPI compilation, CTSM install integrity
1	13	~3s	Case creation, case.setup, xmlchange/xmlquery, NEON site listing
2	14	~100s	Full case.build, NetCDF I/O, cartopy rendering, Dask chunked I/O

Result: 90/90 pass on native arm64.

Component definitions

Most of the modeling components (CTSM, CLM, CIME, DATM, the NEON workflow) are stock NCAR software that any researcher would use. What ExSOIL adds: the multi-architecture container build, the conda-forge compilation environment, the pre-download script for unreliable NCAR servers, and the project-specific analysis notebooks and modules.

Component	What it does	Source	Status
CTSM 5.4	Community Terrestrial Systems Model. Standalone land modeling framework from NCAR. Assembles CLM, CIME, and the NEON workflow into a single release.	NCAR	Working
CLM 6.0	Community Land Model. The Fortran code that simulates soil temperature, moisture, carbon cycling, vegetation, and hydrology. Produces the scientific output.	NCAR	Working

Component	What it does	Source	Status
CIME 6.1	Common Infrastructure for Modeling the Earth. Build and case management: `create_newcase`, `case.setup`, `case.build`, `case.submit`.	NCAR	Working
CAM	Community Atmosphere Model. Simulates weather by solving atmospheric physics equations. Fills the "atmosphere input" slot in full CESM runs.	NCAR	Not in ExSOIL
DATM	Data atmosphere. Fills the same "atmosphere input" slot as CAM, but reads observed weather from files instead of simulating it. CLM receives the same variables through the same coupler interface either way.	NCAR	Working
NEON site surface data	Static description of the land at each tower location: soil type, soil layer depths, vegetation cover, terrain, drainage. One file per site (~56 KB). Does not change over time.	NCAR	Available
NEON tower forcing	Weather measured continuously by tower instruments: temperature, humidity, wind, radiation, precipitation, every 30 minutes. Drives the simulation forward through time. ~150 KB/month per site. Published by NEON (Google Cloud).	NEON	Available

Component	What it does	Source	Status
NEON terrestrial observations	What the instruments measure *in and on* the ground: soil temperature/moisture at depth, eddy covariance fluxes, vegetation structure, phenology, biomass, nutrient cycling. The **ground truth** for model-observation comparison. CTSM includes `download_eval_files` to fetch these; Modeling_Hub has comparison code paths. The plumbing exists but has not been tested in this container.	NEON	Not yet validated
NEON tower workflow	Combines site surface data and tower forcing: downloads both, creates a case, wires DATM. Single command: `run_tower --neon-sites KONZ`.	NCAR	Working
POP2 / MOM6	Ocean models. Not needed for land-only simulations.	NCAR	Not in ExSOIL
CICE / CISM / WW3	Sea ice, ice sheet, and wave models. Not needed for land-only simulations.	NCAR	Not in ExSOIL
MPICH	MPI library for parallel execution. Single core in the container via `mpiexec -n 1`.	conda-forge	Working
Python stack	xarray, cartopy, matplotlib, scipy, pandas, bokeh, panel, JupyterLab.	conda-forge	Working
Pre-download script	Downloads ~6 GB of global input data with retries and fallback (GDEX, SVN, FTP).	ExSOIL	Working
analytics_modules	Kalman filter calibration, model misfit diagnostics, S3 data access.	ExSOIL	Needs validation
Modeling_Hub notebook	Model-data evaluation and Kalman filter calibration.	ExSOIL	Needs validation
Design_Hub_v2 notebook	CLM forcing perturbation experiments, scenario comparison.	ExSOIL	Needs validation